Algebraic Data Types with Sealed Records

ADTs Sealed · Exhaustive Payment describe · Build with Waqar

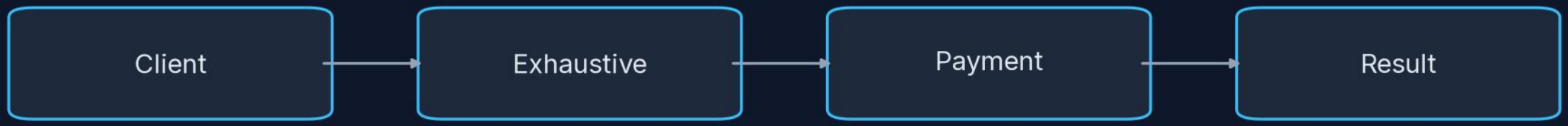
Today's Hook

- Problem: Exhaustive Payment describe
- Pattern: ADTs Sealed — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Exhaustive Payment describe
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



ADTs Sealed: Exhaustive Payment describe

Real-World Scenario

- Running example: Exhaustive Payment describe
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.adt.problem;

import java.util.*;

public final class NaivePay {
  public static String describe(String type, int cents){
    if("card".equals(type)) return "card:"+cents;
    if("cash".equals(type)) return "cash:"+cents;
    return "other";
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.adt.problem;

import java.util.*;
import java.util.function.*;

sealed interface Payment permits Card, Cash, Wire {}
record Card(String last4, int cents) implements Payment {}
record Cash(int cents) implements Payment {}
record Wire(String iban, int cents) implements Payment {}
final class Payments {
  static String describe(Payment p){
    return switch (p) {
      case Card(String last4, int cents) -> "card ****"+last4+" "+cents;
      case Cash(int cents) -> "cash "+cents;
      case Wire(String iban, int cents) -> "wire "+iban+" "+cents;
    };
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.adt.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== ADTs: exhaustive Payment describe ===");
    System.out.println("--- naive ---"); System.out.println(NaivePay.describe("card", 10));
System.out.println("--- ADT ---");
System.out.println(Payments.describe(new Card("1234", 1999)));
System.out.println(Payments.describe(new Cash(500)));
System.out.println(Payments.describe(new Wire("DE89", 10000)));
  }
}
```

Walkthrough Output

```
=== ADTs: exhaustive Payment describe ===
--- naive ---
card:10
--- ADT ---
card ****1234 1999
cash 500
wire DE89 10000
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with ADTs Sealed.

Related Patterns

- Contrast ADTs Sealed with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → ADTs Sealed → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.